

Bait Blocker

V 1.0



Anti-Phishing Analysis Engine

Yehoshua Gruenspecht

<https://github.com/Yehoshua18/BaitBlocker>

Table of Contents

Table of Contents	2
Abstract	3
Introduction	4
The Human Link - Social Engineering	4
Phishing	4
Detection and Prevention	4
Implementation	6
Overall	6
Fast API	6
Layer 1: Cache	7
Layer 2: External API's	8
Layer 3: URL Lexicographical Analysis	8
3.1 Cleanup	9
3.2 Length	9
3.3 Special Characters	9
3.4 Suspicious Keywords	9
3.5 Typosquatting	9
3.6 High Risk TLDs	10
3.7 IP Instead of Domain Name	10
3.8 Random Generated URL	10
Layer 4: Email Text AI Analysis	11
Layer 5: Sandbox Screenshot (Optional)	11
Streamlit UI	11
Future Improvements	12
Appendix	13

Abstract

This report provides a detailed evaluation of Bait Blocker version 1.0, the threat intelligence system I built to combat phishing and social engineering attacks. It walks through the overall architecture, the core detection methods, and how the system blocks malicious links and baiting tactics in real time. I also cover the performance metrics from our testing, deployment approaches for enterprise environments, and the roadmap for future improvements as threats continue to evolve. At its core, this document serves as a technical overview for stakeholders who want to strengthen their organization's defenses against digital deception.

Introduction

To understand why Bait Blocker is necessary, we must first understand why existing security systems such as firewalls and other antivirus software aren't good enough. While other cyber attacks such as XSS or Buffer Overflow can be avoided with proper coding and security measures, phishing can still be effective because it targets the weakest link in the chain of cyber security, the human link.

The Human Link - Social Engineering

At its core, social engineering is not a cyber attack. Instead, social engineering is all about the psychology of persuasion: It targets the mind like your old school grifter or con man. The aim is to gain the trust of targets, so they lower their guard, and then encourage them into taking unsafe actions such as divulging personal information or clicking on web links or opening attachments that may be malicious.¹

Hackers know that breaking into someone's computer to steal their information or money would require passing many levels of security, some of which the user doesn't even know are there. However, tricking a user into willfully giving a hacker money or information would be easier as the levels of defense here are only based on the decisions that the user makes. When it comes to social engineering, all a hacker needs to do is use simple psychology to trick a user into trusting them or feeling a sense of urgency.

Imagine you receive an email from an unknown source. The email has clear grammatical errors and the URL looks like a random string of letters. Obviously, most people wouldn't even see a reason to click on the link and the email will likely be ignored. Now imagine you get an email from Amazon saying you need to transfer money to pay for your monthly membership otherwise your account will be deleted. In this case, most people would get a sense of urgency because they don't want to lose their Amazon account and therefore are likely to miss that Amazon was spelled "amazoon". That's Social Engineering being used to trick users into doing things they don't want to do.

Phishing

Phishing is when attackers send scam emails (or text messages) that contain links to malicious websites. The websites may contain malware (such as ransomware) which can sabotage systems and organisations. Or they might be designed to trick users into revealing sensitive information (such as passwords), or transferring money.²

Bad actors can use social engineering to make phishing attacks as effective as possible.

Detection and Prevention

As mentioned, some of the ways phishing can be detected include but are not limited to:

¹ <https://www.cisco.com/site/us/en/learn/topics/security/what-is-social-engineering.html>

² <https://www.ncsc.gov.uk/guidance/phishing>

- A randomized URL
- Impersonating a real organization
- AI written/poorly written email text
- A sense of urgency created in the email
- Mentions of transferring money
- An unknown sender

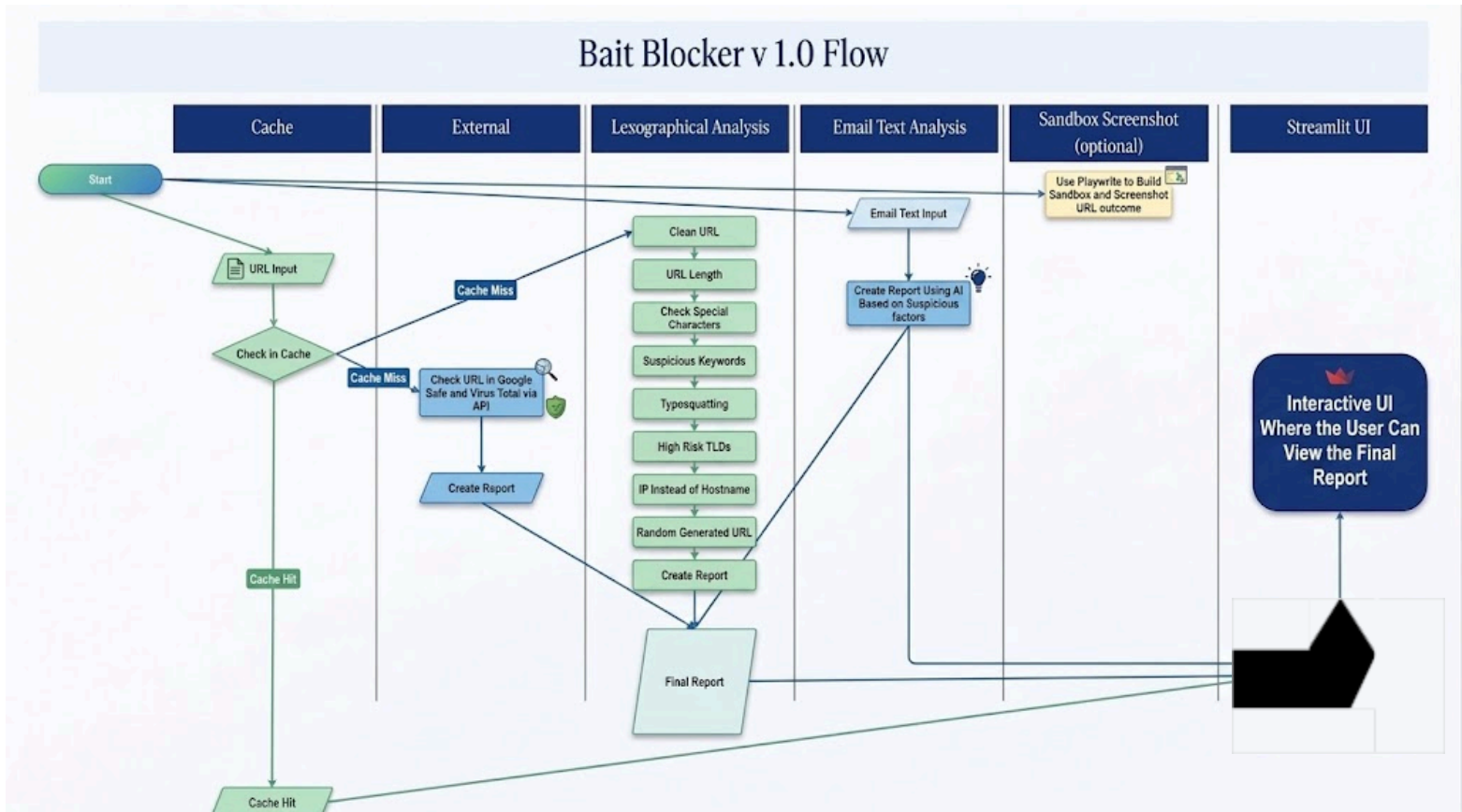
As mentioned, phishing relies on the persuasion of the user to click the link or download a file. Therefore it's up to anyone receiving an email to know what the signs of phishing are to prevent attacks.

That's where Bait Blocker comes in. Bait Blocker uses both existing databases of suspicious links and its own analysis to determine whether an email is suspicious or not.

Implementation

Overall

As mentioned, Bait Blocker uses multiple layers of analysis to determine whether an email is suspicious or not ordered based on runtime of each layer. These layers run asynchronously using FastAPI.



Fast API

In order to run Bait Blocker in an organized way, I used an API and more specifically FastAPI in Python. Bait Blocker is built on a request/response module and runs multiple layers of analysis at once. Therefore, FastAPI was the best choice to use for the backend processing. Using FastAPI, I was able to integrate other existing phishing engines such as Google Safe and Virus Total or Grok for AI email text analyzing. In addition, using FastAPI specifically allows me to also run my own unique logic (such as the lexicographical analyzer) by integrating FastAPI's Pydantic to validate data. FastAPI also creates automatic documentation in the form of SwaggerUI which is great for testing and catching errors.

Layer 1: Cache

Although FastAPI already paves the way for amazing runtime, since bad actors may use the same URL on multiple users, a great way to cut down runtime even more is to use a cache memory system. Using a cache, we can store URLs that Bait Blocker has already seen to return a response automatically without giving it a full analysis and essentially working double.

200

Response body

```
{
  "local_report": {
    "url_lexical_analysis": {
      "url": "https://signin-netflix.xyz",
      "verdict": "High Risk",
      "risk_score": 0.8,
      "reasons": [
        "Blacklisted keywords found: ['netfli",
        "Uses a statistically high-risk top-1"
      ]
    },
    "email_text_analysis": {
      "phishing_probability": 0.8,
      "risk_level": "High",
      "red_flags": [
        "Urgency/Fear",
        "Anomalous Requests"
      ],
      "summary_analysis": "The email creates",
      "recommended_action": "Quarantine"
    }
  },
  "external_report": {
    "status": "received",
    "input_received": "https://signin-netfli",
    "google_safe_browsing": "Clean",
    "risk_level_from_virus_total": "Clean",
    "risk_level_from_virus_total": "Clean",
    "risk_level_from_virus_total": "Clean"
  }
}
```

Response headers

```
content-length: 852
content-type: application/json
date: Tue, 30 Jun 2026 20:16:47 GMT
server: uvicorn
x-cache: MISS
```

Request duration

1547 ms

Server response

Code Details

200

Response body

```
{
  "local_report": {
    "url_lexical_analysis": {
      "url": "https://signin-netflix.xyz",
      "verdict": "High Risk",
      "risk_score": 0.8,
      "reasons": [
        "Blacklisted keywords found: ['netfli",
        "Uses a statistically high-risk top-1"
      ]
    },
    "email_text_analysis": {
      "phishing_probability": 0.8,
      "risk_level": "High",
      "red_flags": [
        "Urgency/Fear",
        "Anomalous Requests"
      ],
      "summary_analysis": "The email creates",
      "recommended_action": "Quarantine"
    }
  },
  "external_report": {
    "status": "received",
    "input_received": "https://signin-netfli",
    "google_safe_browsing": "Clean",
    "risk_level_from_virus_total": "Clean",
    "risk_level_from_virus_total": "Clean",
    "risk_level_from_virus_total": "Clean"
  }
}
```

Response headers

```
content-length: 852
content-type: application/json
date: Tue, 30 Jun 2026 20:17:27 GMT
server: uvicorn
```

Request duration

45 ms

This is evident after running Bait Blocker on the same URL twice we can see a significant drop in runtime without affecting the final output.

Since phishing links tend to only be used for a short amount of time, to utilize memory space efficiently, we will set a TTL (Time To Live) to erase links from the cache after a set time. This set time will be much shorter for clean links to ensure that if attackers hijack the URL, users won't receive a false negative.

Bait Blocker's cache is implemented using FastAPI's cache library. Yet another advantage of FastAPI.

Layer 2: External API's

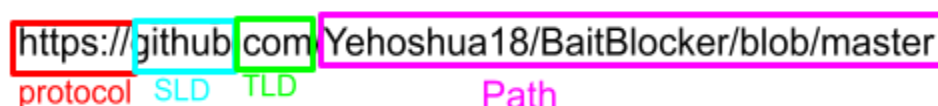
Since I'm not looking to reinvent the wheel and phishing has existed for many years, Bait Blocker takes advantage of these existing databases for a more accurate analysis of suspicious URLs. Since a URL can be suspicious without necessarily being malicious (false positive) or can be malicious without looking suspicious, it would be wise to double check phishing databases to make sure the URL isn't a known phishing link. This is done using Google Safe Browsing and Virus Total API's.

Why both? Google Safe Browsing is completely free and has no limit on the amount of requests. However, it will only return a positive or negative answer unlike Virus Total that returns more information such as how many engines flagged the given URL. The disadvantage with Virus Total is the limited amount of requests (4 per minute or 500 per day). It also takes longer as it reviews many other existing engines although Google notes no guarantees for response time.

Layer 3: URL Lexicographical Analysis

Since the external APIs really aren't enough on their own to determine whether a link is malicious or not, Bait Blocker also relies on a local lexical analysis testing different common phishing tropes.

First we must look at what exactly a URL is to know how to detect a malicious one. **URL** stands for **Uniform Resource Locator**, which literally means "a uniform way to locate resources." Simply put, a URL is the full address used to find and access a web page, file, image, or any other resource on the internet.³ The main component of URLs we will look at for Bait Blocker is the hostname. The hostname consists of the Subdomain to direct to a specific area of a website, the Second Level Domain which is the website's name and the Top Level Domains which describe what kind of website the URL leads to.



https://github.com/Yehoshua18/BaitBlocker/blob/master

protocol SLD TLD Path

For example, www.levnet.ac.il. The subdomain is **www** - world wide web meaning nothing specific. Just the main website. The second level domain (SLD) is **levnet**. Meaning that's the website the URL will take you to. And the TLDs are **ac** - academia and **il** - Israel. Meaning this is a website of an Israeli academic institution.

These components of the URL are the main indicator of the legitimacy of a link. They should be written in clear English and follow a clear pattern. Bait Blocker will look for abnormalities in these areas to ensure the link is secure or at the very least, not suspicious.

³ <https://msf.telkomuniversity.ac.id/en/what-is-a-url-definition-function-and-components-on-the-web/>

3.1 Cleanup

Since hackers are aware of anti-phishing engines, they may change the link in a way that'll try to trick Bait Blocker such as having extra spaces in the URL or using uppercase letters so that it won't get recognized as a suspicious keyword (as will be expanded on in 3.4). Therefore, the first step is to clean up extra spaces and switch all letters to lowercase. Then we will parse the URL to hostname and path. If we're unable to parse, an exception will be thrown as this is extremely suspicious.

3.2 Length

Hackers may try to hide malicious payloads within legit sounding subdomains. For example, instead of the subdomain clearly being fake - yourbank.com, a hacker may be able to hide the domain within legit subdomains - login.secure.yourbank.com. This will look more real to a timid user, however it will force the total length of the URL and specifically the amount of domains to be extra long.

Therefore, Bait Blocker will check the length of the hostname however will not grant a high risk score for excessive length as this can be common amongst legit links as well.

3.3 Special Characters

Since the U in URL stands for uniform, any excessive amount of special characters (specifically hyphens and digits) in a URL will stand out as suspicious. This is especially true for periods as each period indicates another subdomain which standard URLs tend to have at most two or three extra TLDs.

3.4 Suspicious Keywords

Due to phishing relying heavily on social engineering, hackers will try to use different strategies to persuade a user to click on the link sent. This tactic comes in the form of specific words used in the domain name. These keywords can be known brand names that a user will trust such as Google or Facebook. They can also be classic words found in an email regarding a user's personal account information such as 'login' or 'account'.

A user may overlook suspicious details if they are feeling that if they don't act quickly something bad might happen. Therefore, a hacker might use words such as 'urgent' or 'verify' to create a sense of urgency that'll cause a standard user to overlook suspicious details.

To implement a function that checks for these keywords, I first created a database of over one hundred of these kinds of words using the sqlite3 library. For each keyword, the database has a type (financial, urgency, brand etc.) and a risk weight.

I then created a matcher function that scans the database and matches the keywords with the URL, returns the keywords found and the sum of the total risk weight.

3.5 Typosquatting

Bad actors may try to impersonate a real company by using what looks at first glance as the same name. For example, instead of 'google', the domain name might be 'gooogle'. Very similar but not the same.

To calculate how close the fake brand name is to a real brand name I used the Levenshtein distance. Levenshtein distance is a measure of the similarity between two strings, which takes into account the number of insertion, deletion and substitution operations needed to transform one string into the other.⁴ Any edit distance of one or two characters will be seen as highly suspicious.

3.6 High Risk TLDs

Using websites like <https://www.cybercrimeinfocenter.org/phishing-activity-in-tlds-may-july-2025> I was able to determine that most phishing links use similar TLDs that aren't used in legitimate links. By checking the value of the 'endswith' method up against a list of documented TLDs used for phishing, I was able to determine if the TLD should add to the total risk score or not.

3.7 IP Instead of Domain Name

An IP address (Internet Protocol address) is a unique set of numbers assigned to every device connected to the internet. It acts like a digital address that allows devices (like computers, phones, and servers) to locate and communicate with each other online.⁵ Domain names are the "readable for humans" version of IP addresses. Meaning, what we see on our screen as a domain name is actually translated via DNS (Domain Name System) to the IP address of the server that hosts the website.

Instead of creating a unique domain name, hackers may just directly use the IP address of the malicious server to exploit phishing detection that relies on checking the domain name. Using the IP address might also help trick naive users who don't understand what an IP address represents.

To negate this, Bait Blocker checks that the hostname doesn't fit the IPv4 or IPv6 structures.

3.8 Random Generated URL

To cut on costs or to attack multiple people at once, attackers may go for automatic generated URLs. These URLs may consist of both a random string of characters and real life brand names.

To identify such URLs, Bait Blocker uses Shannon's Entropy to calculate the "randomness" (entropy) of the string. Shannon's entropy is defined as:

famous H theorem. We shall call $H = -\sum p_i \log p_i$ the entropy of the set of probabilities $p_1, \dots, p_n$. If x is a chance variable we will write $H(x)$ for its entropy; thus x is not an argument of a function but a label for a number, to differentiate it from $H(y)$ say, the entropy of the chance variable y . 6

In other words, we will calculate the probability of each character in the URL by counting how many times it appears in the URL divided by the total length of the string. Then we will calculate the entropy using Shannon's formula for each probability. If the total entropy is high, that indicates that there is a high level of randomness in the link and we will increase the risk score.

⁴ <https://www.geeksforgeeks.org/dsa/introduction-to-levenshtein-distance/>

⁵ <https://telahosting.ng/blog/how-domain-names-work/>

⁶ A Mathematical Theory of Communication, C.E. Shannon, 1948

Layer 4: Email Text AI Analysis

Since even a link may look clean even after all the layers of defense we built, I used an LLM (Large Language Model) to read the email text and determine whether it is suspicious or not. This was done by looking for specific characteristics found in phishing emails.

Similar to keywords we looked for in the URL, a phishing email will try to persuade a user to click on the link quickly without analyzing thoroughly. Such tactics include impersonating a known brand, asking for money or sensitive information or social engineering in the form of a push for urgency.

As mentioned, attackers may use bots to write thousands of emails at once. Therefore, I also had my LLM look for bot-like typing and add that to the total risk factor.

Layer 5: Sandbox Screenshot (Optional)

The idea behind this feature is to go around the usual defenses entirely and see what would happen if the user were to click on the link? Does the link lead to where the author of the email says it will?

This is done by using the Playwright Python library developed by Microsoft. Playwright is generally used for end to end testing. Playwright Test is an end-to-end test framework for modern web apps. It bundles test runner, assertions, isolation, parallelization and rich tooling. Playwright supports Chromium, WebKit and Firefox on Windows, Linux and macOS, locally or in CI, headless or headed, with native mobile emulation for Chrome (Android) and Mobile Safari.⁷ I personally used playwrights Chromium to mimic Chrome and screenshot the website that the suspicious URL leads to.

While this tool is helpful especially when the attacker uses a real brand in the URL, it drastically increases the runtime. Therefore, if the user is unhappy with the result of Bait Blocker's local and external analyses, he can choose to run the sandbox and see the screenshot.

Streamlit UI

Since Bait Blocker is meant for users who may not be able to spot phishing attacks on their own, I decided to create a more user-friendly UI to make the process easier for less tech-savvy users. The UI doesn't really require so much, especially because I was looking to make it as simple as possible. Therefore, I used Streamlit in Python to make a quick, easy and clean looking web-page with straightforward separate input boxes for the URL and the email text, a multi-level detailed report and a screenshot at the request of the user.

⁷ <https://playwright.dev/docs/intro>

Future Improvements

- **Catch downloads in sandbox.** Since attackers may use phishing to secretly download malware onto your device, a screenshot of the website that the URL leads to might not be enough. A future improvement would be to inform the user if the attacker tried to force a malicious download.
- **WHOIS lookup.** Since most phishing attacks use URLs that were created that day, a WHOIS lookup will give us information about how recently the domain was created and will help Bait Blocker decide if the URL is suspicious. This should also help get around the possibility of a bad actor using Bait Blocker to make sure a URL will be marked clean before using it to attack since it will still get flagged as a recently created domain.

Appendix

<https://www.cybercrimeinfocenter.org/phishing-activity-in-tlds-may-july-2025>

Top rated phishing TLDs

<https://people.math.harvard.edu/~ctm/home/text/others/shannon/entropy/entropy.pdf>

Shannon's entropy paper.